

# Tarefa Textura Visão Computacional e Percepção

Iris Mebs Moraes  
Arthur Dias Viana de Castro

August 2026

## 1 Introdução

A extração de características de textura é uma etapa fundamental em diversas tarefas de visão computacional, como segmentação, classificação e reconhecimento de padrões. Na literatura, os métodos de extração de textura costumam ser agrupados em quatro grandes categorias: estatísticos, estruturais, baseados em transformadas e baseados em modelos [1].

Neste trabalho, optou-se por uma abordagem baseada em filtros, na qual um banco de filtros convolucionais é aplicado sobre a imagem em múltiplas escalas, seguido de uma etapa de agrupamento responsável por separar a imagem em regiões de textura semelhante. As imagens utilizadas nos testes retratam cenários urbanos.

O código-fonte do projeto está disponível no GitHub.

## 2 Metodologia

### 2.1 Organização de entrada e saída

Os arquivos do projeto são organizados em dois diretórios principais. A pasta **images/**, localizada na raiz do projeto, armazena as imagens originais obtidas a partir do álbum compartilhado e utilizadas como entrada para o processamento.

Os resultados são armazenados na pasta **saidas/**, também localizada na raiz. Para cada imagem processada, é criada uma subpasta identificada pelo nome da imagem, contendo os arquivos gerados ao longo do pipeline, como a versão em escala de cinza, os recortes, as segmentações em diferentes escalas e as respostas dos filtros de textura.

Essa organização permite manter separados os dados de entrada e os resultados produzidos, além de facilitar a comparação entre a imagem original e as diferentes etapas do processamento.

### 2.2 Pré-processamento

As imagens foram carregadas do diretório **imagens/** via `cv2.imread` e convertidas para escala de cinza através de `skimage.color.rgb2gray`:

$$I_{gray} = 0.2125 R + 0.7154 G + 0.0721 B \quad (1)$$

Como `cv2.imread` retorna a imagem em BGR e ela não é convertida antes de `rgb2gray`, os canais R e B da Equação 1 ficam trocados no resultado. Em seguida, as imagens foram redimensionadas para  $512 \times 512$  pixels utilizando a função `resize` da biblioteca `skimage.transform`.

Esse pré-processamento padroniza o tamanho e a representação das imagens, reduzindo diferenças de resolução e cor antes da aplicação dos filtros de textura e das etapas de segmentação. Para essas operações, foram utilizadas principalmente as bibliotecas OpenCV, scikit-image e NumPy, responsáveis pela leitura, transformação e manipulação das imagens.

## 2.3 Funções do Sistema

O código é organizado em funções, cada uma é responsável por uma etapa do processamento. A função `carregar_imagens_local()` lê as imagens do diretório `imagens/`. Em seguida, `criar_filtros_textura()` gera os kernels usados para detectar características de textura Sobel X e Y, diagonais de  $45^\circ$  e  $135^\circ$ , LoG, Laplaciano e Gabor nas orientações  $0^\circ$  e  $90^\circ$ , retornando tanto os kernels quanto seus nomes. A função `aplicar_filtros()` recebe uma imagem e essa lista de kernels e devolve a resposta de cada filtro aplicado sobre ela.

A partir dessas respostas, `extrair_vetores_textura_por_escala()` divide a imagem em blocos não sobrepostos e calcula, para cada bloco, a média e o desvio padrão da resposta de cada filtro, montando um vetor de características por bloco. Esses vetores são então agrupados por `agrupar_com_kmeans()`, que implementa o K-Means com  $k = 5$  agrupamentos, semente fixa (`random_state=100`) e 10 inicializações diferentes dos centroides (`n_init=10`), retornando os rótulos de cada bloco e os centroides encontrados. Por fim, `reconstruir_imagem_segmentada()` mapeia cada cluster para uma cor RGB e reconstrói uma imagem colorida com as regiões segmentadas, mantendo as dimensões originais.

## 2.4 Extração de Características

Para caracterizar a textura de cada região da imagem, um banco de 8 filtros convolucionais foi aplicado sobre a imagem em escala de cinza. Os filtros combinam detectores de borda direcionais, detectores de borda isotrópicos e filtros orientados sensíveis a padrões periódicos. A Tabela 1 especifica cada kernel e seu propósito; a Figura 1 mostra os kernels visualizados como mapas de calor.

Considerando o domínio da frequência, os filtros Sobel, LoG e Laplaciano atuam como filtros passa-alta, o filtro Gaussiano da pirâmide atua como passa-baixa, e os filtros Gabor atuam como passa-banda orientados. [2]

Table 1: Especificação dos filtros de textura

| Nome          | Kernel                                                                 | Propósito                                  |
|---------------|------------------------------------------------------------------------|--------------------------------------------|
| Sobel X       | $\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$ | Detecção de bordas verticais               |
| Sobel Y       | $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$ | Detecção de bordas horizontais             |
| Diagonal 45°  | $\begin{pmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{pmatrix}$ | Detecção de padrões diagonais ascendentes  |
| Diagonal 135° | $\begin{pmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{pmatrix}$ | Detecção de padrões diagonais descendentes |
| LoG           | $\nabla^2 G$                                                           | Detecção de bordas circulares              |
| Laplaciano    | $\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$   | Detecção de bordas isotrópica              |
| Gabor 0°      | $g(x, y; \lambda, \theta=0, \psi, \sigma, \gamma)$                     | Textura periódica orientada horizontal     |
| Gabor 90°     | $g(x, y; \lambda, \theta=\pi/2, \psi, \sigma, \gamma)$                 | Textura periódica orientada vertical       |

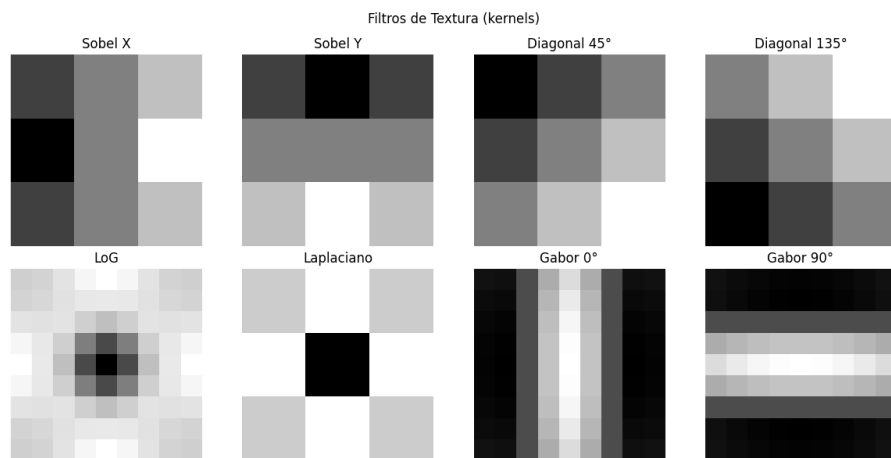


Figure 1: Kernels dos filtros de textura, visualizados como mapas de calor.

## 2.5 Processamento Multi-escala

O processamento constrói uma pirâmide Gaussiana de 3 níveis a partir da imagem redimensionada ( $512 \times 512$ ): a cada nível, aplica-se um filtro Gaussiano para suavização seguido de uma redução das dimensões pela metade (`cv2.pyrDown`). O banco de filtros (Seção 2.4) e a segmentação por K-Means são aplicados independentemente em cada um dos 3 níveis, produzindo uma

segmentação por escala.

O tamanho da janela de agregação (Seção 2.4) também é reduzido pela metade a cada nível, mantendo a proporção entre janela e dimensão da imagem constante entre escalas, garantindo que cada janela sempre cubra a mesma fração relativa da imagem, independente do nível da pirâmide:

Table 2: Tamanhos de janela por nível da pirâmide

| Nível | Dimensão         | Janela         |
|-------|------------------|----------------|
| 1     | $512 \times 512$ | $16 \times 16$ |
| 2     | $256 \times 256$ | $8 \times 8$   |
| 3     | $128 \times 128$ | $4 \times 4$   |

### 3 Resultados

Para cada etapa, plotamos os resultados para acompanhar o processo. A Figura 2 mostra o resultado do pré-processamento sobre uma das imagens, mostrando a conversão para escala de cinza e o redimensionamento para  $512 \times 512$  pixels.



Figure 2: Imagem original e resultado após conversão para escala de cinza e redimensionamento.

As Figuras 3 e 4 mostram o comportamento de cada filtro sobre uma imagem de exemplo na escala 1. Os filtros de borda (Sobel X/Y, diagonais, LoG e Laplaciano) respondem de forma esparsa e concentrada nos contornos, o formato característico dos galhos da árvore e as linhas retas das construções aparecem como traços finos e claros sobre um fundo escuro, coerente com o comportamento de um filtro passa-alta. Já os filtros Gabor em vez de contornos finos, produzem

regiões mais amplas. A fiação elétrica aparece de forma bem mais evidente no Gabor  $90^\circ$ . Já que é uma estrutura fina e predominantemente horizontal.

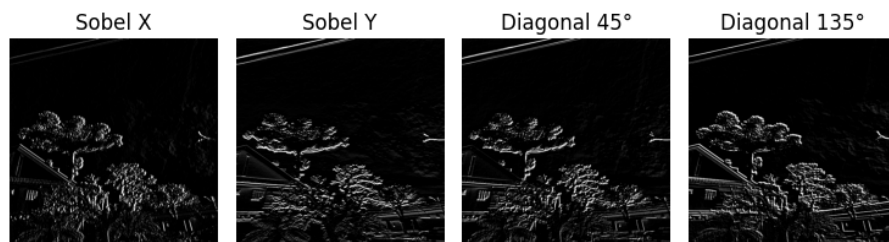


Figure 3: Filtros de borda (Sobel X/Y, diagonais  $45^\circ/135^\circ$ ).

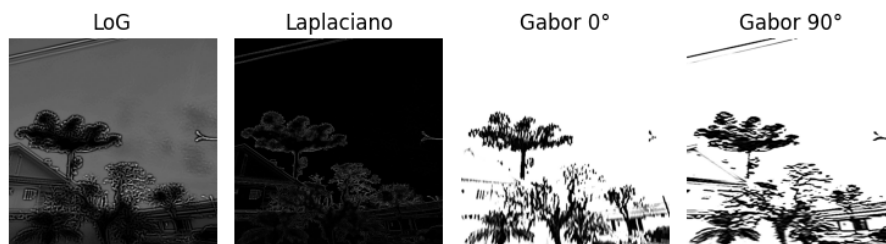


Figure 4: Filtros LoG, Laplaciano e Gabor ( $0^\circ/90^\circ$ ).

A Figura 5 mostra o resultado da segmentação por K-Means ( $k = 5$ ) sobre os vetores de textura extraídos na escala 1 (imagem  $512 \times 512$ , janela  $16 \times 16$ ). Nessa escala, a janela cobre uma região pequena da imagem, favorecendo a captura de detalhes finos de textura, ao custo de uma segmentação mais fragmentada.

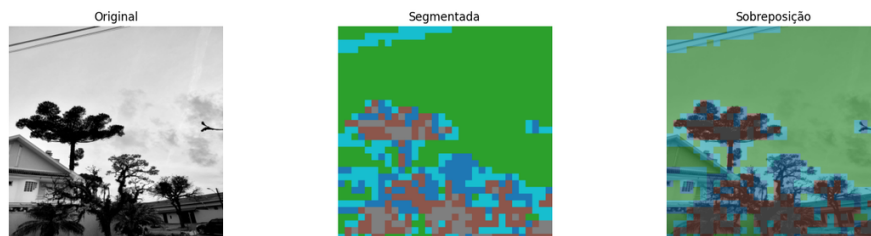


Figure 5: Imagem segmentada com escala 1.

A Figura 6 mostra o mesmo processo aplicado à escala 2 (imagem  $256 \times 256$ , janela  $8 \times 8$ ). Com a redução da imagem pela pirâmide Gaussiana, cada janela passa a cobrir uma fração proporcionalmente maior da textura original,

produzindo regiões segmentadas mais contínuas e com menos ruído do que na escala 1, em troca de menor sensibilidade a detalhes finos.

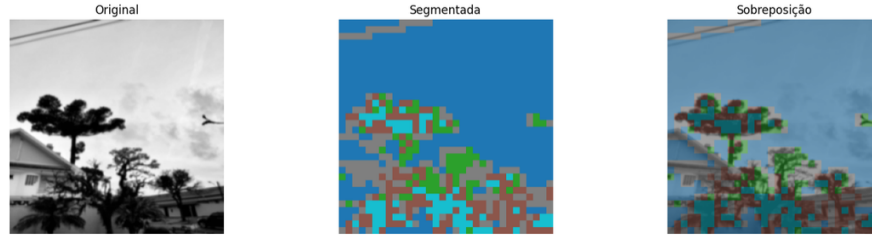


Figure 6: Imagem segmentada com escala 2.

A Figura 7 mostra o resultado na escala 3 (imagem  $128 \times 128$ , janela  $4 \times 4$ ). Por conta da redução feita pela pirâmide Gaussiana cada janela cobre uma fração ainda maior da textura e acaba perdendo detalhes que só aparecem em regiões pequenas.

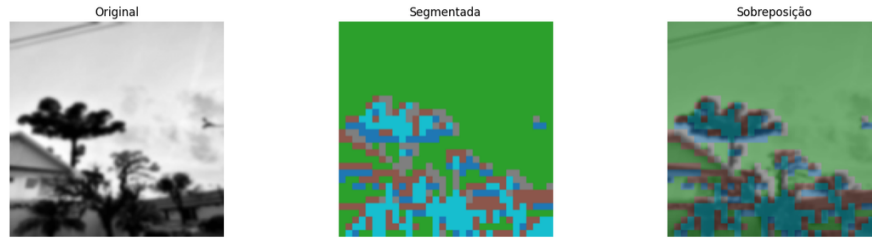


Figure 7: Imagem segmentada com escala 3.

## 4 Conclusão

Os resultados mostram como a escala da imagem altera a resposta do filtro na imagem. Na escala 1, a segmentação capturou detalhes finos de textura mas acaba ficando mais fragmentada, já nas escalas 2 e 3 a segmentação ficou mais contínua, perdendo detalhes pequenos da imagem. Podendo concluir que a escolha da escala está diretamente ligada à característica que deseja capturar. O mesmo pode ser dito sobre o número de clusters: cada imagem, dependendo da quantidade de padrões de textura presentes, provavelmente exigiria um valor diferente de  $k$  para uma segmentação ideal. Fixar  $k = 5$  para todas as imagens é, portanto, uma limitação do método.

A análise das respostas individuais dos filtros também demonstrou a diferença de comportamento entre detectores de borda e filtros Gabor. Os filtros de borda produziram respostas concentradas nos contornos da imagem, já o Gabor respondeu mais a regiões inteiras com padrão repetitivo, como a folhagem da árvore usada no exemplo.

## References

- [1] Anne Humeau-Heurtier. Texture feature extraction methods: A survey. *IEEE Access*, 7:8975–9000, 2019.
- [2] A. Torralba, P. Isola, and W.T. Freeman. *Foundations of Computer Vision*. Adaptive Computation and Machine Learning series. MIT Press, 2024.